



A WORKING REFERENCE

The AI Product Manager's Handbook

The technical foundations, the agent playbook, and the product lessons — written for PMs who have to ship AI, not just talk about it.

By Pulkit Jain

Three parts. A grounding in the technology (LLMs, embeddings, RAG, fine-tuning, prompting, metrics); a playbook for building agents (architectures, tool calling, memory, evaluation); and four deep case studies with the lessons that generalize.

I · The Handbook

II · Agents

III · Case Studies

Current as of mid-2026. The field moves monthly — treat model names, prices, and product facts as dated snapshots and verify before acting.

Contents

0 How to Use This Document

Who it's for · the example callouts · on staying current

1 What Makes AI Products Different

Probabilistic, not deterministic — and what that changes for a PM

PART I · THE AI PM HANDBOOK

2 Large Language Models, Demystified

Tokens · context windows · temperature · hallucination · choosing a model

3 Embeddings & Vector Databases

Meaning as math · semantic search · the vector store

4 Retrieval-Augmented Generation (RAG)

Giving the model your data · the pipeline · where it breaks

5 Fine-Tuning & the Customization Ladder

Prompt → RAG → fine-tune → train: when to climb each rung

6 Prompt Engineering

The techniques that work · system prompts · structure · failure modes

7 AI Product Metrics

Quality, cost, latency, safety · offline vs online · the feedback loop

PART II · BUILDING AI AGENTS

8 What Is an Agent?

Agent vs workflow vs chatbot · the autonomy spectrum

9 Agent Architectures

ReAct · plan-and-execute · multi-agent · the control loop

10 Tool Calling

How agents act on the world · schemas · the orchestration layer

11 Memory Systems

Short-term · long-term · semantic · the context-window problem

12 Evaluation Frameworks

PART III · AI PRODUCT CASE STUDIES

13 ChatGPT

The product that made the model usable

14 Notion AI

AI as a layer on a system of record

15 Perplexity

RAG as a consumer product — the answer engine

16 Cursor

Forking the incumbent to own the workflow

17 Lessons Learned

The patterns that cut across all four

REFERENCE

18 AI PM Checklists

Scoping · building · launching · operating

19 Glossary

The vocabulary, decoded

o How to Use This Document

This is a working reference for product managers who own — or want to own — an AI feature or product. It assumes strong product fundamentals and builds the AI-specific layer on top: enough of the technology to make good decisions and talk credibly to engineers, plus the product patterns that separate AI features that ship and last from demos that don't.

The three parts

Part I is the conceptual toolkit — the things you must understand about how modern AI works to scope it well. **Part II** is the agent playbook, because "agents" is where most ambitious AI roadmaps now point and where the hardest product problems live. **Part III** grounds all of it in four products you already know, extracting the decisions that made them work.

How the callouts work

PM LENS

What you specifically decide or own here, as opposed to engineering or research.

EXAMPLE

A concrete instance — a real product pattern or a worked scenario — because these concepts are slippery in the abstract.

WATCH OUT

A costly, common trap — usually the place where AI behaves differently from the deterministic software you're used to.

NOTE

A definition or piece of context worth keeping in your head.

ON STAYING CURRENT

This field re-invents itself on a quarterly cycle. The concepts here are durable — tokens, retrieval, evaluation, and the autonomy spectrum aren't going anywhere. The specifics — model names, context-window sizes, prices, which company shipped what — are dated to mid-2026 and will drift. Use this to reason; verify the live numbers before you commit.

Traditional software is deterministic: the same input gives the same output, every time, and you can specify the behavior exactly. Modern AI products are **probabilistic**: the same input can give different outputs, the system is built rather than specified, and it fails in ways you can't fully enumerate in advance. Almost every difference for a PM flows from this one shift.

The five shifts

1. **You shape behavior, you don't specify it.** You can't write a requirement that guarantees the output. You steer it — with prompts, data, retrieval, fine-tuning, and guardrails — and then measure how often it lands. Product work becomes an empirical, iterative loop, not a spec handed to engineering.
2. **Quality is a distribution, not a pass/fail.** "Does it work?" becomes "How often, how well, and how badly does it fail when it fails?" You'll live in percentages — accuracy, helpfulness, hallucination rate — not in green/red test results.
3. **The failure modes are weird and open-ended.** Models hallucinate confidently, are sensitive to phrasing, can be manipulated (jailbreaks, prompt injection), and degrade silently when the world shifts under them. You can't unit-test your way to safety.
4. **Cost and latency are product decisions, every call.** Each model call costs money and takes time, and both scale with usage and with how much you ask the model to do. "Make it smarter" often means "make it slower and more expensive." This trade-off is yours to own.
5. **Evaluation is the core competency.** Because you can't specify behavior, your ability to measure it is your steering wheel. Teams that win at AI products are teams that built good evals early (Section 12). Teams that ship on vibes regress silently.

PM LENS

Your mental model shifts from **architect** (specify the system) to **experimentalist** (form a hypothesis, build a version, measure it against a held-out set, iterate). If your roadmap has no line item for "build the eval set," you're flying blind — that's usually the first thing to fix on an AI team.

WATCH OUT — the demo-to-product gap

AI makes **spectacular demos** and **treacherous products**. A demo shows the happy path once; a product faces the long tail of real inputs thousands of times a day, where the 5% failure rate that looked fine in the demo becomes hundreds of bad experiences. The distance from "impressive demo" to "reliable product" is the actual job, and it is mostly invisible to stakeholders who saw the demo and think you're nearly done.

PART I

The AI PM Handbook

The conceptual toolkit: large language models, embeddings and vector search, retrieval-augmented generation, the customization ladder from prompting to fine-tuning, prompt engineering, and the metrics that tell you whether any of it is working.

2

Large Language Models, Demystified

A large language model (LLM) is, at its core, a very sophisticated next-token predictor. Given some text, it produces a probability distribution over what comes next, samples from it, and repeats. Everything impressive it does — reasoning, coding, translation — is an emergent consequence of doing that extremely well at enormous scale. Understanding the handful of mechanics below is enough to make good product calls.

Tokens — the unit of everything

Models don't see words; they see **tokens** — chunks of text, roughly $\frac{3}{4}$ of a word in English. "Unbelievable" might be three tokens; a rare word or a code symbol might be several. Tokens matter to you for two practical reasons: **you pay per token** (input + output), and **context limits are measured in tokens**.

EXAMPLE

A support bot that pastes a customer's entire 40-page contract into every prompt "to be safe" is burning thousands of tokens per message. At scale that's a real line on the bill and added latency — and usually the model only needed two relevant clauses. This is exactly the problem RAG (Section 4) solves.

The context window — the model's working memory

The **context window** is the maximum amount of text (in tokens) the model can consider at once — your prompt, the conversation history, retrieved documents, and the response, all together. Windows have grown enormously (from a few thousand tokens to hundreds of thousands or millions in current frontier models), but they are still finite, and they have a subtler trap.

WATCH OUT — "lost in the middle"

A bigger context window is not free quality. Models attend best to the **start and end** of a long context and can miss information buried in the middle. "Just stuff everything in the prompt" degrades as the prompt grows. **What you put in context, and where, is a design decision** — relevance and ordering beat raw volume.

Temperature & sampling — the creativity dial

Temperature controls randomness. Low temperature (≈ 0) makes the model pick the most likely token almost every time — consistent, focused, good for extraction, classification, and code. High temperature makes it sample more adventurously — varied, creative, good for brainstorming and copy. It's a per-call product knob, not a global setting.

Hallucination — the defining failure

An LLM generates plausible text, not true text. When it doesn't know something, it doesn't say "I don't know" — it confidently invents a plausible-sounding answer: a fake citation, a non-existent API, a wrong date. This **hallucination** isn't a bug to be patched; it's intrinsic to how the model works. You manage it (with retrieval, grounding, constrained outputs, and verification), you don't eliminate it.

Choosing a model — the PM's frame

Dimension	What you're trading
Capability	Frontier models reason better; smaller models are cheaper & faster. Don't use a frontier model to classify a yes/no.
Cost	Priced per million tokens; can vary 10–100× across model tiers. Multiply by your volume before you fall in love with the best model.
Latency	Bigger/reasoning models are slower. A chatbot needs speed; an overnight batch job doesn't.
Context window	How much you can feed it at once.
Hosting & privacy	API (managed, easy) vs open-weights self-hosted (control, data residency, often cheaper at scale).
Modality	Text-only vs multimodal (image, audio, video in/out).

PM LENS

The right answer is usually **a portfolio, not one model**: route easy/high-volume tasks to a small cheap model and reserve the expensive frontier model for the hard cases. "Which model?" is the wrong question; "which model for which task, at what cost and latency?" is the right one. Build the routing logic as a first-class part of the product.

If LLMs are how machines generate language, embeddings are how machines understand similarity of meaning. They are the quiet workhorse behind semantic search, RAG, recommendations, and memory — and they're conceptually simple once you see the trick.

What an embedding is

An **embedding** is a list of numbers (a vector) that represents the meaning of a piece of text, produced by a model trained so that things with similar meaning get similar vectors. "How do I reset my password?" and "I forgot my login credentials" contain almost no shared words, but their embeddings sit close together in this high-dimensional "meaning space." Keyword search misses that; embeddings catch it.

EXAMPLE

Picture a map where every sentence is a point, and distance equals difference in meaning. "King" sits near "queen"; "Paris" sits near "France." To find content related to a user's question, you embed the question, drop it on the map, and grab the nearest points. That nearest-neighbor lookup is the core operation of modern AI retrieval.

Vector databases — search by meaning at scale

Once you've turned a million documents into a million vectors, you need to find the closest ones to a query vector fast. That's what a **vector database** does: it stores embeddings and performs efficient **approximate nearest-neighbor (ANN)** search. Dedicated vector stores and vector features bolted onto familiar databases both exist; the choice is an engineering call, but the PM should know the capability is a building block, not a moonshot.

Concept	Plain-English meaning
Embedding model	Turns text (or images) into vectors. Separate from your generation LLM.
Vector / dimension	The numeric representation; "dimensions" are how long the list is (hundreds to thousands).
Similarity metric	How "closeness" is measured (e.g., cosine similarity).
ANN index	The data structure that makes nearest-neighbor search fast at scale.
Chunking	Splitting documents into passages before embedding — a quality-critical design choice.

WATCH OUT — chunking is product, not plumbing

How you split documents before embedding (by paragraph? by section? with overlap?) hugely affects retrieval quality. Chunks too big dilute the meaning; too small lose context. This looks like an engineering detail but it directly determines whether your RAG system finds the right answer — treat it as a tunable product parameter you measure, not a default you forget.

RAG is the single most important architecture pattern in applied AI today. It solves the two biggest LLM weaknesses at once: models don't know your private/current data, and they hallucinate. The idea is simple — **retrieve the relevant facts first, then ask the model to answer using them.**

Why RAG exists

An LLM's knowledge is frozen at training time and contains nothing about your company's documents, today's data, or this user's account. Fine-tuning to teach it all that is expensive, slow, and still leaks hallucinations. RAG sidesteps the problem: instead of baking knowledge into the model, you **look it up at question time** and hand it to the model as context. The model becomes a reasoning engine over facts you supply, not an oracle relying on memory.

The pipeline

```
# Build time (once, then on updates)
documents → chunk → embed → store in vector DB

# Query time (every user question)
question → embed → search vector DB → top-k relevant chunks
.         → assemble prompt (question + chunks) → LLM → grounded answer (with citations)
```

The model is told, in effect: "Using only the passages below, answer the question; if the answer isn't there, say so." That instruction plus real retrieved context is what makes the answer trustworthy and **citable**.

EXAMPLE

This is exactly how an "answer engine" like Perplexity (Section 15) works: it retrieves live web pages relevant to your query, then has the model synthesize an answer with links back to the sources. And it's how an internal "chat with your docs" tool works — same pipeline, your knowledge base instead of the web. The citations aren't decoration; they're the mechanism that lets a user verify the model didn't make it up.

Where RAG breaks (and what you own)

Failure	Symptom	PM's lever
Bad retrieval	Right answer exists but wasn't fetched	Chunking, embedding model, hybrid keyword+vector search, re-ranking
Stale index	Answers from old data	Re-indexing cadence; freshness SLAs
Context overflow	Too many chunks, model misses the point	Tune top-k; re-rank for relevance
Ungrounded answer	Model ignores context, hallucinates anyway	Stronger grounding prompt; require citations; verify
"I don't know" too often	Over-conservative; coverage gaps	Improve corpus coverage; fall back gracefully

PM LENS

Most "the AI gave a wrong answer" complaints in a RAG product are **retrieval problems, not model problems** — the model answered fine from bad context. Before anyone says "we need a better model," instrument retrieval quality: did we fetch the right passages? That diagnosis saves months. The reflex to upgrade the LLM when the retrieval layer is the culprit is one of the most common and expensive mistakes in applied AI.

5 Fine-Tuning & the Customization Ladder

"Can we train it on our data?" is the most common — and most often premature — request in AI product work. There's a ladder of ways to customize model behavior, from cheap and fast to expensive and slow. The discipline is to climb only as high as you need.

The four rungs

Rung	What it does	Cost / effort	Use when
1 · Prompting	Instruct the base model well; give examples in the prompt	Lowest — minutes	Almost always start here
2 · RAG	Inject relevant knowledge at query time	Low–medium	The model needs your facts or current data
3 · Fine-tuning	Adjust the model's weights on your examples	Medium–high	You need a consistent style, format, or behavior prompting can't reliably get
4 · Pre-training	Train a model from scratch	Enormous	Almost never for a product team

The key distinction: knowledge vs behavior

This is the line that resolves most confusion. **RAG gives the model knowledge; fine-tuning shapes behavior.** If the problem is "it doesn't know our products," that's knowledge → RAG. If the problem is "it knows, but won't answer in our terse, structured house style no matter how I prompt it," that's behavior → fine-tuning. People reach for fine-tuning to inject facts, which it does badly (and the facts go stale instantly); and they reach for prompting to enforce behavior that a fine-tune would nail.

EXAMPLE

A medical-scribe product wants notes in a precise SOAP format every time. Prompting gets it right 80% of the time; the 20% drift is unacceptable to clinicians. Fine-tuning on a few thousand example notes locks in the format reliably — a behavior fix. But the patient-specific facts in each note still come from the transcript at runtime (RAG-like grounding), never from the fine-tune. Knowledge and behavior, handled by the right tools.

WATCH OUT

Fine-tuning is seductive because it sounds like "our own AI." But it adds a training pipeline, data-labeling burden, versioning, and the risk of **catastrophic forgetting** (the model gets worse at general tasks while learning yours). It also has to be redone as base models improve. Exhaust prompting and RAG first; you'll need fine-tuning far less often than your roadmap assumes.

PM LENS

When someone proposes fine-tuning, ask: **"Is this a knowledge gap or a behavior gap?"** and **"Have we exhausted prompting and RAG?"** If it's knowledge, or you haven't, the answer is almost certainly not fine-tuning yet. Climbing the ladder prematurely is one of the most expensive forms of premature optimization in AI.

6 Prompt Engineering

A prompt is the program you write for an LLM — in plain language. Because the model's behavior is so sensitive to how you ask, prompt engineering is a genuine product surface, not a party trick. It's also the cheapest, fastest lever you have, which is why it's always the first one to pull.

The techniques that earn their keep

Technique	What it is & when to use it
Clear instruction	Be specific about role, task, format, and constraints. Most "bad model" output is an underspecified prompt.
Few-shot examples	Show 2-5 input→output examples in the prompt. The single highest-leverage technique for nailing a format or style.
Chain-of-thought	Ask the model to reason step by step before answering. Improves accuracy on multi-step problems (math, logic, planning).
Structured output	Demand JSON / a schema so the output is machine-parseable. Essential when the output feeds other code.
Decomposition	Break a hard task into a chain of simpler prompts rather than one giant ask.
Grounding / "say I don't know"	Instruct the model to use only provided context and to admit ignorance. Core anti-hallucination move.

System prompts vs user prompts

The **system prompt** sets persistent rules, persona, and constraints for the whole interaction ("You are a careful financial assistant; never give specific investment advice; always cite sources"). The **user prompt** is the per-turn request. As a PM you own the system prompt as a product asset — it's where your product's voice, guardrails, and behavior live, and it's versioned, tested, and iterated like any other part of the product.

EXAMPLE

"Summarize this" vs "Summarize this in 3 bullet points for a busy executive, leading with the decision required, in under 60 words." Same model, radically different usefulness. The second is a product decision encoded as a prompt — and it's the kind of thing you A/B test, not guess.

WATCH OUT — prompts are fragile & attackable

Two hazards. First, prompts are brittle: a wording tweak that helps one case can quietly break others — so changes need to run against your eval set, not just the example you were looking at. Second, **prompt injection**: untrusted text (a web page, a user message, a document) can contain instructions that hijack your model ("ignore previous instructions and..."). Never trust that your system prompt is in control when the model is reading attacker-influenced content. Treat prompt changes like code changes and treat external content as hostile.

7 AI Product Metrics

In deterministic software, metrics are about usage and business outcomes. In AI products you add a whole layer beneath that: **is the AI itself any good?** You measure on four axes — quality, cost, latency, and safety — across two settings: offline (before you ship) and online (in production).

The four axes

Axis	Example metrics	Why the PM cares
Quality	Accuracy, helpfulness, faithfulness/groundedness, task success rate, thumbs-up rate	The core question; usually needs an eval set + human or LLM judgment
Cost	Tokens per request, \$ per task, \$ per active user	AI unit economics can sink a product that "works"
Latency	Time-to-first-token, total response time, p95	Slow AI feels broken regardless of quality
Safety	Hallucination rate, harmful-output rate, refusal accuracy, jailbreak resistance	The reputational floor; one bad output can dominate the narrative

Offline vs online

Offline evaluation runs candidate prompts/models against a fixed, curated set of test cases with known good answers — your regression suite for AI (Section 12). It lets you compare versions before shipping.

Online metrics are what real users do: engagement, retention, task completion, edit/acceptance rate (did they keep the AI's output or rewrite it?), escalation rate, and ultimately the business outcome. You need both — offline to iterate safely, online to know it actually helped.

EXAMPLE — the acceptance-rate signal

For an AI code-completion or writing tool, **acceptance rate** (what fraction of suggestions the user keeps) is gold: it's an implicit, high-volume quality signal generated by real work, no survey needed. Cursor and similar tools live and die by it. Pair it with a quality eval so you don't reward suggestions that are accepted but subtly wrong.

WATCH OUT — the vanity-metric trap returns

"Messages sent" or "queries run" can rise while the product gets worse — users re-asking because the first answer failed looks identical to healthy engagement. Always pair an engagement metric with a quality/success metric. High volume driven by retries is a failure wearing a growth costume.

PM LENS

Define your **quality bar as a number on a named eval set** before you build, and put cost-per-task and p95 latency on the same dashboard as your business metrics. The teams that ship reliable AI are the ones for whom "we improved faithfulness from 82% to 91% while holding cost flat" is a normal sentence — not the ones arguing about whether the latest prompt "feels better."

PART II

Building AI Agents

Where most ambitious AI roadmaps now point — and where the hardest product problems live. What an agent actually is, the architectures that power them, how they act on the world through tools, how they remember, and how you keep them honest with evaluation.

8 What Is an Agent?

"Agent" is the most over-used word in AI right now, so pin it down. An **agent** is an LLM-driven system that can decide what to do next — choosing actions, using tools, and looping until a goal is met — rather than running a fixed script. The defining feature is that the **model controls the flow of execution**, not your code.

The spectrum from chatbot to agent

Type	Who controls the flow	Example
Single LLM call	Your code; one in, one out	"Summarize this email"
Chatbot	Your code; multi-turn but linear	A Q&A assistant
Workflow / chain	Your code; predefined steps, LLM fills them	"Extract → classify → route" pipeline
Agent	The model decides steps, tools, and when to stop	"Research this company and draft a memo" — it chooses what to search, read, and write

NOTE — workflow vs agent is a real choice, not a status game

A **workflow** (you define the steps; the LLM executes each) is more predictable, cheaper, easier to debug, and right for most problems. An **agent** (the model improvises the steps) is more flexible but less predictable, more expensive, and harder to control. "Agentic" is not automatically better. The skill is choosing the least-autonomous design that solves the problem.

PM LENS

Default to the **lowest autonomy that works**. Every increment of autonomy you hand the model buys flexibility at the cost of predictability, latency, cost, and debuggability. Start with a constrained workflow; promote to a full agent only where the task genuinely can't be scripted because the steps depend on what's discovered along the way.

Under the hood, almost every agent is a loop: the model looks at the situation, decides on an action, the action runs, the result comes back, and it loops again until done. The architectures differ mainly in how much the agent plans ahead and how many agents are involved.

The core loop

```
# The agent loop, in essence
while not done:
    thought = model.reason(goal, history, observations)
    action = model.choose_action(thought) # e.g. call a tool
    result = execute(action) # run it in the world
    history.append(action, result) # remember what happened
    done = model.is_goal_met(history)
```

The common patterns

Pattern	How it works	Trade-off
ReAct (reason + act)	Interleaves reasoning and tool use step by step: think, act, observe, repeat	Flexible & transparent; can wander or loop without limits
Plan-and-execute	Makes a full plan first, then executes the steps	More structured & cheaper; brittle if reality diverges from the plan
Reflection / self-critique	Generates, critiques its own output, revises	Higher quality; more calls, more cost & latency
Multi-agent	Several specialized agents (e.g., researcher + writer + reviewer) coordinate	Powerful for complex jobs; coordination overhead & failure-mode explosion

EXAMPLE — a coding agent in the loop

Ask a coding agent to "fix the failing test." It reasons ("I should read the test"), acts (reads the file), observes the error, reasons again ("the function returns the wrong type"), acts (edits the code), runs the test, sees it pass, and stops. No human scripted those steps — the agent chose each one based on what it found. That's the ReAct loop doing real work, and it's roughly how tools like Cursor's agent mode operate (Section 16).

WATCH OUT — runaway loops & compounding errors

Agents can loop forever, repeat failed actions, or take a wrong early step that poisons everything after it (errors **compound** across steps — 90% reliability per step is ~35% over ten steps). You must design hard stops: max iterations, budgets (time, tokens, money), and human-in-the-loop checkpoints for consequential actions. An unbounded agent is a production incident waiting to happen.

PM LENS

Two product decisions are yours, not engineering's: **where the human approves** (which actions require a confirm before they execute — sending the email, making the payment, deleting the data) and **what the guardrails are** (max steps, spend caps, allowed tools). These define the agent's "blast radius." Decide them deliberately, per action, by consequence.

10 Tool Calling

A language model alone can only produce text. **Tool calling** (a.k.a. function calling) is what lets it act — search the web, query a database, send a message, run code, hit an API. It is the bridge between "a model that talks" and "a system that does things," and it's the foundation every agent is built on.

How it works

You describe the available tools to the model — each with a name, a description, and a schema for its inputs. When the model decides a tool is needed, instead of answering in prose it emits a **structured request** ("call `get_weather` with `{city: 'Mumbai'}`"). Your code runs the real function, returns the result to the model, and the model continues with that new information. The model never executes anything itself — it requests, your system executes. That separation is the key safety boundary.

```
# You expose a tool the model can request
tool: get_order_status
  description: "Look up the status of a customer order"
  input: { order_id: string }

# Model emits a call → your code runs it → result goes back to the model
model → call get_order_status({order_id: "A-1029"})
your_code → returns {status: "shipped", eta: "Jun 20"}
model → "Your order shipped and arrives June 20."
```

NOTE — MCP, the emerging standard

The **Model Context Protocol (MCP)** has become a common open standard for connecting models to tools and data sources in a reusable way — think of it as a universal adapter so tools built once can be used across many AI apps. When teams talk about "connectors" or "integrations" for an AI product, increasingly they mean MCP.

What the PM owns here

Decision	Why it matters
Which tools exist	The tools define what the agent can do. This is your product's capability surface.
Tool descriptions	The model picks tools based on their descriptions — vague descriptions cause wrong tool use. This is prompt engineering for tools.
Permissions & scoping	Read-only vs write; which data each tool can touch. The agent's power is the sum of its tools.
Confirmation gates	Which tool calls run automatically vs require human approval.
Error handling	What the agent sees when a tool fails — so it can recover instead of hallucinating.

WATCH OUT — the lethal trifecta

Be very careful when an agent simultaneously has (1) access to **private data**, (2) exposure to **untrusted content** (web pages, emails, documents), and (3) the ability to **communicate externally**. That combination lets a prompt-injection attack read your secrets and exfiltrate them. If your design has all three, you need explicit isolation and human gates — this is one of the defining security problems of agentic products.

LLMs are **stateless** — each call starts from nothing; the model has no inherent memory of your last message. Everything it “remembers” is text you re-supply in the context window. Memory systems are the engineering around that limitation, and they’re what make an assistant feel continuous instead of amnesiac.

The types of memory

Type	What it holds	How it's done
Short-term (working)	The current conversation	The context window — recent turns passed back each call
Long-term (semantic)	Durable facts about the user/world (“prefers metric units”)	Stored externally, retrieved via embeddings (RAG over memory)
Episodic	Past events/interactions (“last week you booked X”)	Logged and selectively recalled
Procedural	Learned skills/instructions the system reuses	Saved prompts, tools, routines

The context-window problem

You can’t just keep appending the whole history forever — it overflows the window and costs explode. So memory becomes a **curation problem**: what do you keep verbatim, what do you summarize, what do you store externally and retrieve only when relevant? Common techniques: **summarize** older turns into a running synopsis, **retrieve** only the relevant past via embeddings, and **extract** durable facts into a structured profile.

EXAMPLE

A long-running assistant doesn’t re-read your entire chat history on every message. It keeps the last few turns in full, a running summary of earlier context, and a small store of extracted facts (“works in healthcare, based in India, prefers concise answers”). On a new message it pulls the few relevant memories into context — not everything. That selective recall is what makes it feel like it knows you without drowning in tokens.

WATCH OUT — memory is a privacy & trust surface

The moment your product “remembers” users, you’ve created a sensitive data store with real questions: what’s retained, for how long, can the user see and delete it, and what happens when it remembers something wrong or stale? A confidently-recalled incorrect memory is worse than no memory. Build inspection and deletion in from day one — it’s a product requirement, not a setting.

This is the most under-appreciated and most decisive capability in AI product work. Because you can't specify behavior, **your ability to measure it is your only steering wheel**. Teams that build good evals early ship reliably and improve fast; teams that ship on vibes regress silently and can't tell whether a change helped.

What an eval actually is

An **eval** is a test suite for non-deterministic systems: a curated set of inputs, plus a way to judge whether the output is good. It's your regression suite — run it on every prompt change, model swap, or new release to catch quality drops before users do. The hard part isn't running evals; it's **defining "good"** for an open-ended output.

Ways to judge output

Method	How it works	Best for
Exact / rule-based	Compare to a known answer or check a rule (valid JSON? contains the figure?)	Structured tasks, classification, extraction
Human evaluation	People rate outputs on a rubric	The gold standard for nuanced quality; slow & costly
LLM-as-judge	A strong model scores outputs against criteria	Scalable proxy for human judgment; needs validation against humans
A/B testing	Compare versions on real users & real outcomes	The final word — did it actually help in production

Evaluating agents: trajectories, not just answers

For agents, the final answer isn't enough — you evaluate the **trajectory**: did it choose the right tools, in a sensible order, without wasteful or dangerous steps? An agent can reach a correct answer through a reckless path (it deleted a file along the way) or a wasteful one (40 steps for a 3-step task). Good agent evals score the process, not only the outcome.

EXAMPLE — the eval set as a moat

The most valuable asset a mature AI team owns is often its **curated eval set** — hundreds of real, hard cases with agreed-upon "good" answers, including the embarrassing failures customers reported. It's what lets them swap to a new model in an afternoon and know whether quality went up or down. Competitors can copy your prompt; they can't easily copy years of accumulated, labeled edge cases.

PM LENS

Make the eval set a **first-class deliverable you personally drive**. Seed it from real user failures, grow it continuously, and treat "we added 20 cases from this week's complaints" as normal hygiene. Refuse to approve prompt or model changes that haven't been run against it. This one habit separates AI teams that compound from ones that thrash.

PART III

AI Product Case Studies

Four products you already use, read as product decisions. Each made a distinctive bet about how to turn a capable model into something people pay for. The figures are mid-2026 snapshots; the lessons outlast the numbers.

13 ChatGPT

ChatGPT — OpenAI

The product that made the model usable. Bet: wrap a powerful model in the simplest possible interface and let the world find the uses.

~1B

monthly active users (2026)

~900M+

weekly active users

~\$25B

annualized revenue

~50M

consumer subscribers

The product insight

The underlying model existed (as an API) before ChatGPT, used mostly by developers. The breakthrough wasn't a smarter model — it was **packaging**: a free, zero-friction chat box that anyone could type into and immediately get value, with no prompt expertise required. It reframed a developer tool as a consumer product, and reached 100 million users in about two months — the fastest consumer-app ramp in history at the time.

What it got right (the PM lessons)

- **Distribution through simplicity.** A chat box is the most universal interface imaginable. Zero learning curve was the feature.
- **Conversation as the unit.** Multi-turn dialogue let users refine and recover from bad answers, hiding the model's unreliability behind a forgiving interface.
- **Free tier as the funnel.** Mass free usage created the data, the habit, and the word-of-mouth that fed a paid tier and an enterprise business on top.
- **Rapid surface expansion.** From a chat box it grew into voice, vision, tools, custom GPTs, and an enterprise platform — each riding the same habit.

LESSON

The model is rarely the product. ChatGPT's lesson is that **packaging and friction reduction can matter more than capability**. The same model behind an API nobody outside engineering touched became, behind a chat box, the fastest-growing consumer product in history.

THE FLIP SIDE

That very generality is now its strategic challenge: an open chat box has no inherent moat, and rivals reach users through workflows ChatGPT doesn't own. Its app-market share has slipped even as absolute usage grows — a reminder that being first and general isn't the same as being defensible.

14 Notion AI

Notion AI — Notion

AI as a layer on a system of record. Bet: don't build a new AI app — embed AI where the work and the data already live.

Embedded

in an existing workspace

Bundled

into Business (~\$20/seat)

Model-agnostic

(multi-model)

Agents + Enterprise Search

The product insight

Notion didn't make users go to a separate AI tool and copy-paste context back and forth. It put AI **inside the documents and databases people already maintain** — so the AI has the user's actual content as context for free, and the user never breaks flow. The data and the distribution were already there; AI became a feature layer, not a new product.

What it got right (the PM lessons)

- **Proprietary context is the moat.** The AI is grounded in your workspace — your notes, your projects, your wiki. That context is something a standalone chatbot can't replicate, and it's defensible.
- **AI in the workflow, not beside it.** Summarize this page, draft from this database, ask a question across the whole workspace — the value lands at the point of work.
- **Evolution to agents & search.** It grew from "help me write" into autonomous agents that do multi-step work across pages and **Enterprise Search** that answers across Notion plus connected tools (Slack, Drive, GitHub) — turning the workspace into an answer engine over your own knowledge.
- **Model-agnostic by design.** Users can pick the underlying model; Notion owns the experience and the data, not the model — insulating it from any single model vendor.

LESSON

If you already own a system of record with users and data, **AI is a feature you add, not a company you start**. Embedding AI where the data and the workflow already live beats a standalone tool that has to acquire both from scratch. Distribution + proprietary context is a stronger position than a clever model.

THE TENSION

The pricing is the friction: meaningful AI sits behind the higher tier, and usage-metered agents add variable cost — a recurring complaint. Monetizing an embedded AI layer without alienating the base is an unsolved balancing act, and a live lesson in AI pricing.

Perplexity — the Answer Engine

RAG as a consumer product. Bet: people don't want ten blue links — they want a cited answer.

~\$20B

valuation (2026)

~\$450–500M

ARR

~30–45M

monthly users

Lean

team, high revenue/employee

The product insight

Perplexity is RAG (Section 4) turned into a mainstream product. Instead of relying on the model's frozen, hallucination-prone memory, it **retrieves live web sources for every query and synthesizes a direct answer with citations**. The citations are the whole trick: they make an LLM answer trustworthy and verifiable, solving the hallucination problem that makes raw chatbots risky for factual lookup.

What it got right (the PM lessons)

- **A sharp wedge against a giant.** It didn't try to out-search Google generally; it owned **high-intent research questions** with a better answer format. Focus beat breadth.
- **Citations as a trust mechanism.** Sourcing every claim turned the model's biggest weakness into a visible strength — and a defensible product behavior.
- **Multi-model routing.** Rather than betting on one model, it routes queries across models — staying capability-current without being hostage to a single vendor.
- **Subscription-first conviction.** It leaned into paid subscriptions (and pushed into agentic browsing) rather than chasing ad pennies, betting on loyal power users over mass monetization — high revenue per employee with a famously lean team.

LESSON

You don't beat an incumbent by copying it — you **re-frame the job**. Perplexity reframed "search" as "get an answer," and made RAG's citation pattern the trust mechanism that an open chatbot lacks. A focused wedge with a defensible behavior beats a broad assault.

THE RISK

Its product depends on synthesizing content publishers create — and publishers increasingly resist AI consumption of their work. The business sits on a contested supply of source content; the revenue-share "publisher program" is its attempt to defuse a structural dependency. A reminder that an AI product's data supply chain is a strategic risk, not a given.

Cursor — Anysphere

Forking the incumbent to own the workflow. Bet: meet developers exactly where they work, then make the IDE agentic.

\$2B+

ARR (2026)

~\$50B

valuation (in talks)

1M+

paying customers

~67–70%

of Fortune 500

The product insight

Cursor didn't build a new editor from scratch or a sidebar plugin. It **forked VS Code** — inheriting the entire ecosystem of extensions, themes, and muscle memory developers already had — so switching cost was near zero, and then layered deep AI in: predictive multi-line **Tab** completion and an agent that can read and edit across the whole codebase. By owning the editor, it owns the developer's primary work surface, and it became the fastest application-layer SaaS to \$1B and then \$2B ARR.

What it got right (the PM lessons)

- **Inherit distribution by forking the incumbent.** Forking VS Code meant instant familiarity and zero migration pain — it imported its own onboarding.
- **Own the surface, not just a feature.** A plugin lives at the host's mercy; being the IDE let Cursor build AI experiences a plugin couldn't, and capture the whole workflow.
- **Depth where it counts.** Tab completion and codebase-aware agents delivered visceral daily value — measured by **acceptance rate** (Section 7), the implicit quality signal it optimizes relentlessly.
- **Land developers, expand to enterprise.** Bottom-up product-led adoption by individual developers became top-down enterprise deals — the now-dominant motion for technical AI tools.

LESSON

When an incumbent owns the workflow, sometimes the move is to **fork it, not flank it** — absorb its ecosystem and switching costs, then out-build it on the new (AI) dimension. Owning the work surface lets you ship experiences a bolt-on feature never could, and it's a far stronger position than a plugin renting space in someone else's product.

THE EXPOSURE

Cursor's product rides on frontier models it largely doesn't own (though it now trains its own too), and the foundation-model labs are building competing coding tools directly. Sitting atop someone else's model while they eye your market is the structural tension of every application-layer AI company — strategically central, not a footnote.

Read together, the four cases rhyme. The patterns below show up again and again in AI products that work — and their absence shows up in the ones that don't.

Pattern	What it means	Seen in
The model is not the product	Packaging, workflow fit, and trust mechanics decide success more than raw model quality	All four
Distribution & context beat the model	Owning where users already work — and the data there — is the durable moat	Notion, Cursor
Solve hallucination visibly	Citations / grounding turn the model's weakness into a trust feature	Perplexity
Reduce friction to near zero	The easiest possible entry (chat box, a fork, in-workflow) is itself the growth engine	ChatGPT, Cursor
Re-frame the job, don't copy	Win against incumbents by changing what the task is, not doing theirs slightly better	Perplexity
Stay model-agnostic	Own the experience & data; treat models as swappable to avoid vendor lock-in	Notion, Perplexity
Land-and-expand	Free/individual adoption → paid → enterprise is the dominant AI motion	ChatGPT, Cursor
Mind the supply chain	Your dependence on models and on source data is a strategic risk to manage	Cursor, Perplexity

THE SYNTHESIS

If there's one sentence to carry from Part III: **capability is table stakes; the defensible product is built around the model — in the workflow you own, the data you ground in, the trust you engineer, and the friction you remove.** Every team can call the same models. What differs is everything around the call. That's where the PM works.

The pages to return to. Adapt to your product and stage.

18.1 · Scoping an AI feature

- ☐ Confirmed AI is the right tool — not deterministic code that would be cheaper and reliable.
- ☐ Defined the task precisely and what a "good" output looks like, in writing.
- ☐ Chosen the lowest rung of the customization ladder that could work (prompt → RAG → fine-tune).
- ☐ Picked the model(s) by capability × cost × latency — not just "the best one."
- ☐ Estimated cost-per-task and latency at expected volume before building.
- ☐ Identified the worst-case failure and its consequence.

18.2 · Building

- ☐ Built a seed eval set of real cases with agreed "good" answers before iterating on prompts.
- ☐ Versioned the system prompt as a product asset; changes run against the eval set.
- ☐ For RAG: instrumented retrieval quality separately from generation quality.
- ☐ For agents: set max steps, budgets, and human-approval gates by action consequence.
- ☐ Scoped tool permissions (read vs write; data access) to the minimum needed.
- ☐ Checked for the lethal trifecta (private data + untrusted content + external comms).
- ☐ Designed graceful failure: "I don't know," fallbacks, and escalation paths.

18.3 · Quality & safety before launch

- ☐ Set a numeric quality bar on the named eval set; current version clears it.
- ☐ Measured hallucination/faithfulness, not just helpfulness.
- ☐ Red-teamed for prompt injection, jailbreaks, and harmful outputs.
- ☐ Confirmed grounding/citations where users need to verify claims.
- ☐ Reviewed data handling, retention, and user deletion for any memory/PII.
- ☐ Stress-tested the long tail, not just the demo happy path.

18.4 · Launch & operate

- ☐ Instrumented online quality (acceptance/edit rate, task success), not just usage.
- ☐ Paired every engagement metric with a quality/success metric (retry ≠ growth).
- ☐ Put cost-per-task and p95 latency on the same dashboard as business metrics.
- ☐ Set up a pipeline to capture real failures and feed them back into the eval set.
- ☐ Established a cadence to re-test against newer/cheaper models as they ship.
- ☐ Defined an incident path for a bad-output event (detect → contain → communicate → fix).

The vocabulary, decoded.

Models & mechanics

LLM — Large Language Model; a model that predicts text token by token and, at scale, reasons, writes, and codes.

Token — The unit a model reads/writes (~¾ of a word). You pay per token and context limits are measured in them.

Context window — The max tokens a model can consider at once (prompt + history + retrieved data + output).

Temperature — The randomness dial; low = focused/consistent, high = varied/creative.

Hallucination — Confident, plausible, false output. Intrinsic to LLMs; managed, not eliminated.

Multimodal — A model that handles more than text (image, audio, video) as input and/or output.

Inference — Running the model to get an output (as opposed to training it).

Open-weights — A model whose parameters are released, so you can self-host and customize it.

Retrieval & customization

Embedding — A vector representing the meaning of text, so similar meanings sit close together.

Vector database — A store optimized for fast nearest-neighbor search over embeddings.

ANN Approximate Nearest Neighbor — The fast similarity-search technique behind vector stores.

Chunking — Splitting documents into passages before embedding; a quality-critical design choice.

RAG Retrieval-Augmented Generation — Retrieve relevant facts, then have the model answer using them. Grounds answers & cuts hallucination.

Grounding — Constraining the model to answer from supplied/retrieved sources.

Re-ranking — Re-ordering retrieved results by relevance before passing them to the model.

Fine-tuning — Adjusting a model's weights on your examples to shape behavior, style, or format.

Pre-training — Training a model from scratch; enormous cost, almost never a product-team task.

Customization ladder — Prompt → RAG → fine-tune → pre-train; climb only as high as needed.

Prompting

Prompt — The plain-language instruction you give a model; effectively its program.

System prompt — Persistent rules, persona, and constraints for the whole interaction; a product asset.

Few-shot — Putting example input→output pairs in the prompt to steer format/style.

Chain-of-thought — Prompting the model to reason step by step before answering.

Structured output — Forcing a machine-parseable format (e.g., JSON) so output can feed other code.

Prompt injection — An attack where untrusted content hijacks the model's instructions.

Jailbreak — Tricking a model into bypassing its safety constraints.

Agents

Agent — An LLM-driven system where the model decides the steps, chooses tools, and loops toward a goal.

Workflow / chain — Predefined steps the LLM fills in; more predictable than an agent.

Tool / function calling — Letting a model request actions (search, query, send) that your code executes.

MCP Model Context Protocol — An open standard for connecting models to tools and data sources.

ReAct — An agent pattern interleaving reasoning and action step by step.

Plan-and-execute — Plan all steps first, then carry them out.

Reflection — An agent critiquing and revising its own output for quality.

Trajectory — The full sequence of an agent's steps; what you evaluate, not just the final answer.

Lethal trifecta — Private data + untrusted content + external comms in one agent; an exfiltration risk.

Memory & evaluation

Stateless — Models don't remember between calls; memory is text you re-supply.

Short-term / working memory — The current conversation held in context.

Long-term / semantic memory — Durable facts stored externally and retrieved when relevant.

Eval (evaluation set) — A curated test suite for judging non-deterministic output; your AI regression suite.

LLM-as-judge — Using a strong model to score outputs at scale; validate against humans.

Acceptance / edit rate — How often users keep vs rewrite AI output; a strong implicit quality signal.

Faithfulness / groundedness — Whether an answer is supported by its sources.

p95 latency — The response time 95% of requests beat; a truer UX measure than the average.

Closing note. The concepts here are the durable core of AI product work; the model names, prices, and company figures are mid-2026 snapshots that will drift. Use this to reason and to ask sharper questions — then verify the live specifics before you decide. The job, underneath all the churn, stays the same: build the defensible product around the model.